Menú, pausa y game over - estados del juego

Objetivo: Al terminar puedes organizar el juego en estados (menú, jugando, pausa, fin) con una variable y cambiar de uno a otro con eventos de teclado.
Antes de empezar:

- Lección 5 - el juego con choques, puntos y vidas.

Para copiar el código: al copiar desde un PDF, muchos visores pierden los espacios del principio de cada línea, y en Python esos espacios importan. En la misma carpeta de esta guía está cada paso como archivo `.py`: ábrelo y copia de ahí.

Paso a paso

Menú, pausa y game over

Ahora mismo, cuando pierdes, la ventana se cierra de golpe; un juego de verdad no hace eso. La idea de hoy es una sola variable, state, que dice en qué momento del juego estás. Con ella, al perder vas a ver una pantalla de fin. Vas a tener un menú para empezar, y para volver a jugar. Y vas a poder pausar con la tecla P.

Lo que necesitan los estados - una fuente grande, una función para centrar texto y la variable state

Vamos por partes. Arriba, junto a la fuente que ya tenías, agrega una fuente más grande, big_font, de ochenta puntos, para los mensajes. Después de la clase Enemy, agrega la función draw_centered. Recibe un texto, una fuente y una posición vertical, y lo dibuja centrado de lado a lado. Y antes del bucle, crea la variable state con el valor playing. Esa variable va a decir en qué momento del juego estás.

Tu `game.py` queda así — las líneas resaltadas son las nuevas o cambiadas (archivo `paso-1.py`):

```
import random

import pygame

pygame.init()
screen = pygame.display.set_mode((800, 600))
pygame.display.set_caption("Most Wanted UFO Mini")
clock = pygame.time.Clock()

background = pygame.image.load("assets/background_4.png").convert()
player_image = pygame.image.load("assets/player_1.png").convert_alpha()
bullet_image = pygame.image.load("assets/Bullet.gif").convert_alpha()
enemy_image = pygame.image.load("assets/enemy_1.png").convert_alpha()
shoot_sound = pygame.mixer.Sound("assets/shoot.wav")
hit_sound = pygame.mixer.Sound("assets/collision.wav")
lose_sound = pygame.mixer.Sound("assets/loselife.wav")
```

```
font = pygame.font.Font(None, 36)
big_font = pygame.font.Font(None, 80)


class Enemy(pygame.sprite.Sprite):
    def __init__(self):
        super().__init__()
        self.image = enemy_image
        self.rect = self.image.get_rect()
        self.reset()

    def reset(self):
        self.rect.midbottom = (random.randint(20, 780), random.randint(-500, -40))
        self.speed = random.randint(1, 2)

    def update(self):
        self.rect.y += self.speed


def draw_centered(text, font, y):
    image = font.render(text, True, (255, 255, 255))
    screen.blit(image, image.get_rect(center=(400, y)))


player = player_image.get_rect(midbottom=(400, 590))
speed = 6
bullets = []
cooldown = 0
enemies = pygame.sprite.Group(Enemy() for _ in range(8))
score = 0
lives = 3
state = "playing"

running = True
while running:
    for event in pygame.event.get():
        if event.type == pygame.QUIT:
            running = False

    keys = pygame.key.get_pressed()
    if keys[pygame.K_LEFT]:
        player.x -= speed
    if keys[pygame.K_RIGHT]:
        player.x += speed
    player.clamp_ip(screen.get_rect())

    if cooldown > 0:
        cooldown -= 1
    if keys[pygame.K_SPACE] and cooldown == 0:
        bullets.append(bullet_image.get_rect(midbottom=player.midtop))
        shoot_sound.play()
        cooldown = 15

    for bullet in bullets:
        bullet.y -= 10
    bullets = [b for b in bullets if b.bottom > 0]
    enemies.update()

    for enemy in enemies:
        hit = enemy.rect.collidelist(bullets)
```

```
        if hit != -1:
            bullets.pop(hit)
            enemy.reset()
            score += 10
            hit_sound.play()
        elif enemy.rect.top > 600 or enemy.rect.colliderect(player):
            enemy.reset()
            lives -= 1
            lose_sound.play()
    if lives <= 0:
        running = False

    screen.blit(background, (0, 0))
    enemies.draw(screen)
    for bullet in bullets:
        screen.blit(bullet_image, bullet)
    screen.blit(player_image, player)
    hud = font.render(f"Score: {score}   Lives: {lives}", True, (255, 255, 255))
    screen.blit(hud, (10, 10))
    pygame.display.flip()
    clock.tick(60)

pygame.quit()
```

Con if state == "playing", la lógica del juego solo corre mientras se juega (la sangría la mete en el if)

Ahora viene el cambio que parece grande. Escribe if state == "playing", y mueve debajo, cuatro espacios a la derecha, todo lo que va desde las flechas hasta los choques; los eventos y el dibujo quedan afuera. Las líneas en verde son las mismas de antes, solo que corridas. Así, cuando state deje de ser playing, nada de esto se mueve. Lo primero es el movimiento con las flechas. Sigue la cadencia de disparo, también corrida: el cooldown baja en cada cuadro, y si llegó a cero y pulsas la barra espaciadora, sale una bala nueva, suena el disparo, y el cooldown vuelve a quince. Después, las balas suben, se borran las que salieron por arriba, y el grupo de enemigos se mueve con update. Por último, los choques: igual que en la lección anterior, una bala que acierta suma diez puntos, y un enemigo que te toca o se escapa te quita una vida. El único cambio de verdad está al final: cuando las vidas llegan a cero, ya no pones running en False, sino que cambias state a game_over.

Tu `game.py` queda así — las líneas resaltadas son las nuevas o cambiadas (archivo `paso-2.py`):

```
import random

import pygame

pygame.init()
screen = pygame.display.set_mode((800, 600))
pygame.display.set_caption("Most Wanted UFO Mini")
clock = pygame.time.Clock()

background = pygame.image.load("assets/background_4.png").convert()
player_image = pygame.image.load("assets/player_1.png").convert_alpha()
bullet_image = pygame.image.load("assets/Bullet.gif").convert_alpha()
enemy_image = pygame.image.load("assets/enemy_1.png").convert_alpha()
shoot_sound = pygame.mixer.Sound("assets/shoot.wav")
hit_sound = pygame.mixer.Sound("assets/collision.wav")
lose_sound = pygame.mixer.Sound("assets/loselife.wav")
```

```
font = pygame.font.Font(None, 36)
big_font = pygame.font.Font(None, 80)


class Enemy(pygame.sprite.Sprite):
    def __init__(self):
        super().__init__()
        self.image = enemy_image
        self.rect = self.image.get_rect()
        self.reset()

    def reset(self):
        self.rect.midbottom = (random.randint(20, 780), random.randint(-500, -40))
        self.speed = random.randint(1, 2)

    def update(self):
        self.rect.y += self.speed


def draw_centered(text, font, y):
    image = font.render(text, True, (255, 255, 255))
    screen.blit(image, image.get_rect(center=(400, y)))


player = player_image.get_rect(midbottom=(400, 590))
speed = 6
bullets = []
cooldown = 0
enemies = pygame.sprite.Group(Enemy() for _ in range(8))
score = 0
lives = 3
state = "playing"

running = True
while running:
    for event in pygame.event.get():
        if event.type == pygame.QUIT:
            running = False

    if state == "playing":
        keys = pygame.key.get_pressed()
        if keys[pygame.K_LEFT]:
            player.x -= speed
        if keys[pygame.K_RIGHT]:
            player.x += speed
        player.clamp_ip(screen.get_rect())

        if cooldown > 0:
            cooldown -= 1
        if keys[pygame.K_SPACE] and cooldown == 0:
            bullets.append(bullet_image.get_rect(midbottom=player.midtop))
            shoot_sound.play()
            cooldown = 15

        for bullet in bullets:
            bullet.y -= 10
        bullets = [b for b in bullets if b.bottom > 0]
        enemies.update()

        for enemy in enemies:
```

```
            hit = enemy.rect.collidelist(bullets)
            if hit != -1:
                bullets.pop(hit)
                enemy.reset()
                score += 10
                hit_sound.play()
            elif enemy.rect.top > 600 or enemy.rect.colliderect(player):
                enemy.reset()
                lives -= 1
                lose_sound.play()
        if lives <= 0:
            state = "game_over"

    screen.blit(background, (0, 0))
    enemies.draw(screen)
    for bullet in bullets:
        screen.blit(bullet_image, bullet)
    screen.blit(player_image, player)
    hud = font.render(f"Score: {score}   Lives: {lives}", True, (255, 255, 255))
    screen.blit(hud, (10, 10))
    pygame.display.flip()
    clock.tick(60)

pygame.quit()
```

La pantalla de fin se dibuja según el valor de state

Falta mostrarlo. Al final del dibujo, justo antes de display.flip, agrega: si state es game_over, usa draw_centered para escribir GAME OVER con la fuente grande, y debajo el puntaje.

Tu `game.py` queda así — las líneas resaltadas son las nuevas o cambiadas (archivo `paso-3.py`):

```
import random

import pygame

pygame.init()
screen = pygame.display.set_mode((800, 600))
pygame.display.set_caption("Most Wanted UFO Mini")
clock = pygame.time.Clock()

background = pygame.image.load("assets/background_4.png").convert()
player_image = pygame.image.load("assets/player_1.png").convert_alpha()
bullet_image = pygame.image.load("assets/Bullet.gif").convert_alpha()
enemy_image = pygame.image.load("assets/enemy_1.png").convert_alpha()
shoot_sound = pygame.mixer.Sound("assets/shoot.wav")
hit_sound = pygame.mixer.Sound("assets/collision.wav")
lose_sound = pygame.mixer.Sound("assets/loselife.wav")
font = pygame.font.Font(None, 36)
big_font = pygame.font.Font(None, 80)


class Enemy(pygame.sprite.Sprite):
    def __init__(self):
        super().__init__()
        self.image = enemy_image
        self.rect = self.image.get_rect()
        self.reset()
```

```
    def reset(self):
        self.rect.midbottom = (random.randint(20, 780), random.randint(-500, -40))
        self.speed = random.randint(1, 2)

    def update(self):
        self.rect.y += self.speed


def draw_centered(text, font, y):
    image = font.render(text, True, (255, 255, 255))
    screen.blit(image, image.get_rect(center=(400, y)))


player = player_image.get_rect(midbottom=(400, 590))
speed = 6
bullets = []
cooldown = 0
enemies = pygame.sprite.Group(Enemy() for _ in range(8))
score = 0
lives = 3
state = "playing"

running = True
while running:
    for event in pygame.event.get():
        if event.type == pygame.QUIT:
            running = False

    if state == "playing":
        keys = pygame.key.get_pressed()
        if keys[pygame.K_LEFT]:
            player.x -= speed
        if keys[pygame.K_RIGHT]:
            player.x += speed
        player.clamp_ip(screen.get_rect())

        if cooldown > 0:
            cooldown -= 1
        if keys[pygame.K_SPACE] and cooldown == 0:
            bullets.append(bullet_image.get_rect(midbottom=player.midtop))
            shoot_sound.play()
            cooldown = 15

        for bullet in bullets:
            bullet.y -= 10
        bullets = [b for b in bullets if b.bottom > 0]
        enemies.update()

        for enemy in enemies:
            hit = enemy.rect.collidelist(bullets)
            if hit != -1:
                bullets.pop(hit)
                enemy.reset()
                score += 10
                hit_sound.play()
            elif enemy.rect.top > 600 or enemy.rect.colliderect(player):
                enemy.reset()
                lives -= 1
                lose_sound.play()
```

```
        if lives <= 0:
            state = "game_over"

    screen.blit(background, (0, 0))
    enemies.draw(screen)
    for bullet in bullets:
        screen.blit(bullet_image, bullet)
    screen.blit(player_image, player)
    hud = font.render(f"Score: {score}   Lives: {lives}", True, (255, 255, 255))
    screen.blit(hud, (10, 10))
    if state == "game_over":
        draw_centered("GAME OVER", big_font, 250)
        draw_centered(f"Score: {score}", font, 340)
    pygame.display.flip()
    clock.tick(60)

pygame.quit()
```

Con el estado game_over el bucle sigue pero la partida se detiene

Ejecuta el juego, y esta vez no toques nada. Los enemigos pasan, y pierdes las vidas una por una. Al llegar a cero, aparece GAME OVER: el bucle sigue corriendo, pero ya no mueve nada, porque state no es playing.

Pruébalo:

```
python game.py
```

Un evento de teclado cambia de estado y reinicia la partida

Ahora el menú. Cambia el valor inicial de state: en vez de playing, ahora es menu. Así, al abrir el juego, nada se mueve todavía, porque la lógica solo corre cuando state es playing. En los eventos, agrega un caso nuevo: KEYDOWN, que llega una sola vez cada vez que pulsas una tecla. Si es Enter y no estás jugando ni en pausa, un estado que agregarás al final, empieza una partida desde cero. O sea, solo desde el menú o la pantalla de fin: state pasa a playing, el ovni vuelve abajo al centro, los puntos a cero, tres vidas, sin balas, y cada enemigo vuelve arriba con reset. Si olvidas algo, la partida nueva arrastra datos de la anterior.

Tu `game.py` queda así — las líneas resaltadas son las nuevas o cambiadas (archivo `paso-4.py`):

```
import random

import pygame

pygame.init()
screen = pygame.display.set_mode((800, 600))
pygame.display.set_caption("Most Wanted UFO Mini")
clock = pygame.time.Clock()

background = pygame.image.load("assets/background_4.png").convert()
player_image = pygame.image.load("assets/player_1.png").convert_alpha()
bullet_image = pygame.image.load("assets/Bullet.gif").convert_alpha()
enemy_image = pygame.image.load("assets/enemy_1.png").convert_alpha()
shoot_sound = pygame.mixer.Sound("assets/shoot.wav")
hit_sound = pygame.mixer.Sound("assets/collision.wav")
lose_sound = pygame.mixer.Sound("assets/loselife.wav")
```

```
font = pygame.font.Font(None, 36)
big_font = pygame.font.Font(None, 80)


class Enemy(pygame.sprite.Sprite):
    def __init__(self):
        super().__init__()
        self.image = enemy_image
        self.rect = self.image.get_rect()
        self.reset()

    def reset(self):
        self.rect.midbottom = (random.randint(20, 780), random.randint(-500, -40))
        self.speed = random.randint(1, 2)

    def update(self):
        self.rect.y += self.speed


def draw_centered(text, font, y):
    image = font.render(text, True, (255, 255, 255))
    screen.blit(image, image.get_rect(center=(400, y)))


player = player_image.get_rect(midbottom=(400, 590))
speed = 6
bullets = []
cooldown = 0
enemies = pygame.sprite.Group(Enemy() for _ in range(8))
score = 0
lives = 3
state = "menu"

running = True
while running:
    for event in pygame.event.get():
        if event.type == pygame.QUIT:
            running = False
        elif event.type == pygame.KEYDOWN:
            if event.key == pygame.K_RETURN and state not in ("playing", "paused"):
                state = "playing"
                player.midbottom = (400, 590)
                score = 0
                lives = 3
                bullets = []
                for enemy in enemies:
                    enemy.reset()

    if state == "playing":
        keys = pygame.key.get_pressed()
        if keys[pygame.K_LEFT]:
            player.x -= speed
        if keys[pygame.K_RIGHT]:
            player.x += speed
        player.clamp_ip(screen.get_rect())

        if cooldown > 0:
            cooldown -= 1
        if keys[pygame.K_SPACE] and cooldown == 0:
            bullets.append(bullet_image.get_rect(midbottom=player.midtop))
```

```
            shoot_sound.play()
            cooldown = 15

        for bullet in bullets:
            bullet.y -= 10
        bullets = [b for b in bullets if b.bottom > 0]
        enemies.update()

        for enemy in enemies:
            hit = enemy.rect.collidelist(bullets)
            if hit != -1:
                bullets.pop(hit)
                enemy.reset()
                score += 10
                hit_sound.play()
            elif enemy.rect.top > 600 or enemy.rect.colliderect(player):
                enemy.reset()
                lives -= 1
                lose_sound.play()
        if lives <= 0:
            state = "game_over"

    screen.blit(background, (0, 0))
    enemies.draw(screen)
    for bullet in bullets:
        screen.blit(bullet_image, bullet)
    screen.blit(player_image, player)
    hud = font.render(f"Score: {score}   Lives: {lives}", True, (255, 255, 255))
    screen.blit(hud, (10, 10))
    if state == "game_over":
        draw_centered("GAME OVER", big_font, 250)
        draw_centered(f"Score: {score}", font, 340)
    pygame.display.flip()
    clock.tick(60)

pygame.quit()
```

Lo que se dibuja depende del estado - el título en el menú, el juego en los demás

Ahora el dibujo. El fondo se dibuja siempre. Después, si state es menu, usas draw_centered para escribir el nombre del juego con la fuente grande y, debajo, que pulses Enter para jugar. Si no, en cualquier otro estado, dibujas lo mismo de antes: los enemigos, las balas, el ovni y el marcador, ahora cuatro espacios a la derecha, dentro del else. Así, en la pantalla de fin, el juego se sigue viendo detrás. Por último, la pantalla de fin: junto al puntaje, ahora dice que pulses Enter para jugar de nuevo, porque desde ahí también puedes empezar otra partida.

Tu `game.py` queda así — las líneas resaltadas son las nuevas o cambiadas (archivo `paso-5.py`):

```
import random

import pygame

pygame.init()
screen = pygame.display.set_mode((800, 600))
pygame.display.set_caption("Most Wanted UFO Mini")
clock = pygame.time.Clock()
```

```
background = pygame.image.load("assets/background_4.png").convert()
player_image = pygame.image.load("assets/player_1.png").convert_alpha()
bullet_image = pygame.image.load("assets/Bullet.gif").convert_alpha()
enemy_image = pygame.image.load("assets/enemy_1.png").convert_alpha()
shoot_sound = pygame.mixer.Sound("assets/shoot.wav")
hit_sound = pygame.mixer.Sound("assets/collision.wav")
lose_sound = pygame.mixer.Sound("assets/loselife.wav")
font = pygame.font.Font(None, 36)
big_font = pygame.font.Font(None, 80)


class Enemy(pygame.sprite.Sprite):
    def __init__(self):
        super().__init__()
        self.image = enemy_image
        self.rect = self.image.get_rect()
        self.reset()

    def reset(self):
        self.rect.midbottom = (random.randint(20, 780), random.randint(-500, -40))
        self.speed = random.randint(1, 2)

    def update(self):
        self.rect.y += self.speed


def draw_centered(text, font, y):
    image = font.render(text, True, (255, 255, 255))
    screen.blit(image, image.get_rect(center=(400, y)))


player = player_image.get_rect(midbottom=(400, 590))
speed = 6
bullets = []
cooldown = 0
enemies = pygame.sprite.Group(Enemy() for _ in range(8))
score = 0
lives = 3
state = "menu"

running = True
while running:
    for event in pygame.event.get():
        if event.type == pygame.QUIT:
            running = False
        elif event.type == pygame.KEYDOWN:
            if event.key == pygame.K_RETURN and state not in ("playing", "paused"):
                state = "playing"
                player.midbottom = (400, 590)
                score = 0
                lives = 3
                bullets = []
                for enemy in enemies:
                    enemy.reset()

    if state == "playing":
        keys = pygame.key.get_pressed()
        if keys[pygame.K_LEFT]:
            player.x -= speed
        if keys[pygame.K_RIGHT]:
```

```
            player.x += speed
        player.clamp_ip(screen.get_rect())

        if cooldown > 0:
            cooldown -= 1
        if keys[pygame.K_SPACE] and cooldown == 0:
            bullets.append(bullet_image.get_rect(midbottom=player.midtop))
            shoot_sound.play()
            cooldown = 15

        for bullet in bullets:
            bullet.y -= 10
        bullets = [b for b in bullets if b.bottom > 0]
        enemies.update()

        for enemy in enemies:
            hit = enemy.rect.collidelist(bullets)
            if hit != -1:
                bullets.pop(hit)
                enemy.reset()
                score += 10
                hit_sound.play()
            elif enemy.rect.top > 600 or enemy.rect.colliderect(player):
                enemy.reset()
                lives -= 1
                lose_sound.play()
        if lives <= 0:
            state = "game_over"

    screen.blit(background, (0, 0))
    if state == "menu":
        draw_centered("Most Wanted UFO Mini", big_font, 250)
        draw_centered("Press ENTER to play", font, 340)
    else:
        enemies.draw(screen)
        for bullet in bullets:
            screen.blit(bullet_image, bullet)
        screen.blit(player_image, player)
        hud = font.render(f"Score: {score}   Lives: {lives}", True, (255, 255, 255))
        screen.blit(hud, (10, 10))
    if state == "game_over":
        draw_centered("GAME OVER", big_font, 250)
        draw_centered(f"Score: {score}  -  ENTER to play again", font, 340)
    pygame.display.flip()
    clock.tick(60)

pygame.quit()
```

La pausa es otro estado

La pausa es solo un estado más. Debajo del caso de Enter, agrega dos casos: si pulsas la tecla P mientras juegas, state pasa a paused; si la pulsas en pausa, vuelve a playing. Van con elif, para que en cada pulsación corra uno solo: con dos if sueltos, el segundo te sacaría de la pausa al instante. Y usas KEYDOWN porque llega una vez por pulsación; con get_pressed, la pausa parpadearía en cada cuadro. Al final, después de dibujar el juego, si state es paused, escribe PAUSED con la fuente grande en el centro de la pantalla. Como va después, queda encima de la escena congelada.

Tu `game.py` queda así — las líneas resaltadas son las nuevas o cambiadas (archivo `paso-6.py`):

```
import random

import pygame

pygame.init()
screen = pygame.display.set_mode((800, 600))
pygame.display.set_caption("Most Wanted UFO Mini")
clock = pygame.time.Clock()

background = pygame.image.load("assets/background_4.png").convert()
player_image = pygame.image.load("assets/player_1.png").convert_alpha()
bullet_image = pygame.image.load("assets/Bullet.gif").convert_alpha()
enemy_image = pygame.image.load("assets/enemy_1.png").convert_alpha()
shoot_sound = pygame.mixer.Sound("assets/shoot.wav")
hit_sound = pygame.mixer.Sound("assets/collision.wav")
lose_sound = pygame.mixer.Sound("assets/loselife.wav")
font = pygame.font.Font(None, 36)
big_font = pygame.font.Font(None, 80)


class Enemy(pygame.sprite.Sprite):
    def __init__(self):
        super().__init__()
        self.image = enemy_image
        self.rect = self.image.get_rect()
        self.reset()

    def reset(self):
        self.rect.midbottom = (random.randint(20, 780), random.randint(-500, -40))
        self.speed = random.randint(1, 2)

    def update(self):
        self.rect.y += self.speed


def draw_centered(text, font, y):
    image = font.render(text, True, (255, 255, 255))
    screen.blit(image, image.get_rect(center=(400, y)))


player = player_image.get_rect(midbottom=(400, 590))
speed = 6
bullets = []
cooldown = 0
enemies = pygame.sprite.Group(Enemy() for _ in range(8))
score = 0
lives = 3
state = "menu"

running = True
while running:
    for event in pygame.event.get():
        if event.type == pygame.QUIT:
            running = False
        elif event.type == pygame.KEYDOWN:
            if event.key == pygame.K_RETURN and state not in ("playing", "paused"):
                state = "playing"
                player.midbottom = (400, 590)
```

```
                score = 0
                lives = 3
                bullets = []
                for enemy in enemies:
                    enemy.reset()
            elif event.key == pygame.K_p and state == "playing":
                state = "paused"
            elif event.key == pygame.K_p and state == "paused":
                state = "playing"

    if state == "playing":
        keys = pygame.key.get_pressed()
        if keys[pygame.K_LEFT]:
            player.x -= speed
        if keys[pygame.K_RIGHT]:
            player.x += speed
        player.clamp_ip(screen.get_rect())

        if cooldown > 0:
            cooldown -= 1
        if keys[pygame.K_SPACE] and cooldown == 0:
            bullets.append(bullet_image.get_rect(midbottom=player.midtop))
            shoot_sound.play()
            cooldown = 15

        for bullet in bullets:
            bullet.y -= 10
        bullets = [b for b in bullets if b.bottom > 0]
        enemies.update()

        for enemy in enemies:
            hit = enemy.rect.collidelist(bullets)
            if hit != -1:
                bullets.pop(hit)
                enemy.reset()
                score += 10
                hit_sound.play()
            elif enemy.rect.top > 600 or enemy.rect.colliderect(player):
                enemy.reset()
                lives -= 1
                lose_sound.play()
        if lives <= 0:
            state = "game_over"

    screen.blit(background, (0, 0))
    if state == "menu":
        draw_centered("Most Wanted UFO Mini", big_font, 250)
        draw_centered("Press ENTER to play", font, 340)
    else:
        enemies.draw(screen)
        for bullet in bullets:
            screen.blit(bullet_image, bullet)
        screen.blit(player_image, player)
        hud = font.render(f"Score: {score}   Lives: {lives}", True, (255, 255, 255))
        screen.blit(hud, (10, 10))
    if state == "paused":
        draw_centered("PAUSED", big_font, 300)
    if state == "game_over":
        draw_centered("GAME OVER", big_font, 250)
        draw_centered(f"Score: {score}  -  ENTER to play again", font, 340)
```

```
    pygame.display.flip()
    clock.tick(60)

pygame.quit()
```

Recorrer todos los estados

Ejecuta el juego. Empieza en el menú; con Enter arranca la partida. Ahora pulsa P: todo se congela, y aparece PAUSED. Pulsa P otra vez, y el juego sigue justo donde estaba.

Pruébalo:

```
python game.py
```

Lo que aprendiste

Repasemos. state dice en qué momento del juego estás. La lógica del juego solo corre cuando state es playing. KEYDOWN sirve para acciones de una sola vez, como empezar o pausar; get_pressed, para lo que se mantiene, como moverse. Y el menú, la pausa y el fin son solo estados distintos. Tu ejercicio: guarda el mejor puntaje, y muéstralo en el menú. En la última lección llega el jefe final, y harás tu propio mod.

- state dice en qué momento del juego estás
- La lógica solo corre en playing
- KEYDOWN para acciones de una vez
- Menú, pausa y fin son estados

Resumen

- Una variable `state` (`"menu"`, `"playing"`, `"paused"`, `"game_over"`) dice en qué momento está el juego.
- La lógica (mover, disparar, chocar) va dentro de `if state == "playing":`; el bucle sigue corriendo en los demás estados.
- `event.type == pygame.KEYDOWN` llega una sola vez al pulsar la tecla: sirve para empezar o pausar. `get_pressed()` sirve para lo que se mantiene apretado.
- Al empezar una partida se reinician `score`, `lives`, `bullets` y los enemigos.
- `draw_centered(texto, fuente, y)` centra un texto con `image.get_rect(center=(400, y))`.
- Para correr muchas líneas a la vez dentro de un `if`, selecciónalas en el editor y pulsa Tab.

Ejercicio

Parte del `game.py` de esta lección.

- Crea `best = 0` junto a `score`.
- Cuando la partida termina (`state = "game_over"`), si `score` es mayor que `best`, guárdalo en `best`.
- Muestra `Best: {best}` en el menú y en la pantalla de fin.
- Bonus: agrega la tecla `Escape` para volver al menú desde la pausa.

Solución

```
best = 0
# ...
# dentro de if state == "playing":
    if lives <= 0:
        state = "game_over"
        best = max(best, score)
# al dibujar el menú y el fin:
draw_centered(f"Best: {best}", font, 400)
# en los eventos:
elif event.key == pygame.K_ESCAPE and state == "paused":
    state = "menu"
```

Gráficos y sonidos: Most Wanted UFO Classic, de Jose Daniel Soto, licencia CC BY 4.0.